



Temps i memòria

Algorísmica

Grau d'Enginyeria Informàtica

Mireia Ribera, ribera@ub.edu



Complexitat dels algoritmes: Temps i memòria

- 1 Concepte de complexitat. Mesura empírica.. Gran O
- 2 Cost de treballar amb text
- 3 Algorismes recursius i pila d'execució
- 4 Dividir i vèncer i teorema Mestre



Com sabem si un algorisme és eficient?

Un algorisme és **eficient** si utilitza el nombre mínim de recursos (cicles de càlcul de l'ordinador i memòria de l'ordinador) possibles.

Fer servir algorismes eficients sempre és adient i moltes vegades és una necessitat ateses les dimensions del problema.

Cost, complexitat, passos computacionals i O gran

En aquesta assignatura ens centrarem sobretot en els recursos de temps i parlarem de **complexitat dels algorismes**, però en alguns casos considerarem els costos de memòria. Sempre buscarem algorismes de complexitat baixa.

Imaginem una funció que processa el senyal d'un sensor de soroll.

Si l'ordinador ha de fer 6 operacions per cada registre d'un sensor... quantes operacions ha de fer per TOTS els sensors en UN DIA COMPLET?



Mesurarem dues coses:

- ❑ **els passos que ha de fer un algorisme** fins arribar a la solució, i
- ❑ **com augmenten** aquests passos quan augmenta la grandària de les dades d'entrada.

An analog clock face is visible on the left side of the slide, showing the numbers 0, 5, and 10. The clock has a red needle pointing towards the 10.

Mesura empírica del cost d'un algorisme

També podem mesurar el cost d'un algorisme de manera empírica, comptant els segons que tarda en executar una funció

```
import time  
  
t0 = time.perf_counter()  
  
# el teu codi  
  
t1 = time.perf_counter()  
  
print(f"Exemple 1 → Temps: {(t1 - t0)*1e6:.8f} \u03BCs")
```

Saber el temps que ha gastat ja ens dona una primera idea del “cost”, però aquesta mesura empírica pot variar d'un ordinador a un altre, i en el mateix ordinador d'un moment a un altre (segons la càrrega d'operacions que estigui realitzant en altres tasques).

Per això voldrem calcular la complexitat de manera independent de l'ordinador.

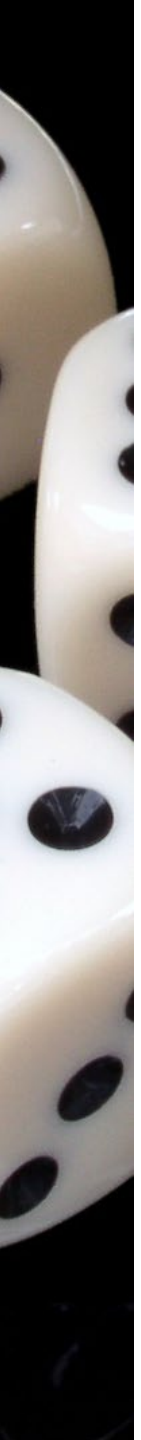
Passos computacionals (1)

Considerarem que l'ordinador fa **un pas** computacional cada vegada que fa **una instrucció simple**: emmagatzema un enter o un caràcter a memòria, fa una comparació o una suma simples... Per exemple:

```
a:int = 5           # 1 pas
for i in range(3):  # El de dintre x 3 (0, 1, 2)
    a = a + 2        # 2 passos, suma + emmagatzematge
print(a)            # 1 pas
```

>11

Aquest codi fa 8 passos.



Passos computacionals (2)

Atenció! L'ordinador a cada pas només pot “operar” amb dades de 64 bits (o 32 bits si és molt antic).

Per això si les dades són més grans, hauríem de reconsiderar els càlculs de l'anterior diapositiva.

Passos computacionals: nombres grans

Les operacions amb nombres grans ja no són tan barates!

```
import math

a:int = 1234585127527575235234982374598245
b:int = 8112387512759287512875851285789127
for i in range(3):
    a += math.sqrt(a+b)
print(a)
```

En les operacions amb nombres molt grans tindrem en compte la mida del nombre, ho veurem al tema dels algoritmes numèrics.

Complexitat segons la mida de l'entrada

Per poder comparar uns algoritmes amb els altres i tenir una idea dels passos computacionals que executaran quan les dades creixin, i per això **mesurem els passos en funció de la grandària de l'entrada**.

```
nums:list[int] = [1, 2, 3, 4, 5]
for i in nums:
    print(i)
```

Quants passos fa? 1 per cada element de la llista. Si al nombre d'elements de la llista li diem n , aquest codi fa n passos.

```
nums:list[int] = [1, 2, 3, 4, 5]
for i in nums:
    for j in nums:
        print(i, j, end=",")
```

Quants passos fa? n per cada element
Si al nombre d'elements de la llista li diem n , aquest codi fa $n \times n$ passos, n^2 .

I aquest codi?

```
nums = [1, 2, 3, 4, 5]
for i in nums:
    print(2 * i)
```

Complexitat: Gran O

En alguns algorismes el cost depèn dels valors de l'entrada, i podríem considerar el millor cas, el cas mig o el pitjor cas. A Algorísmica considerarem només **el pitjor cas, la Gran O**.

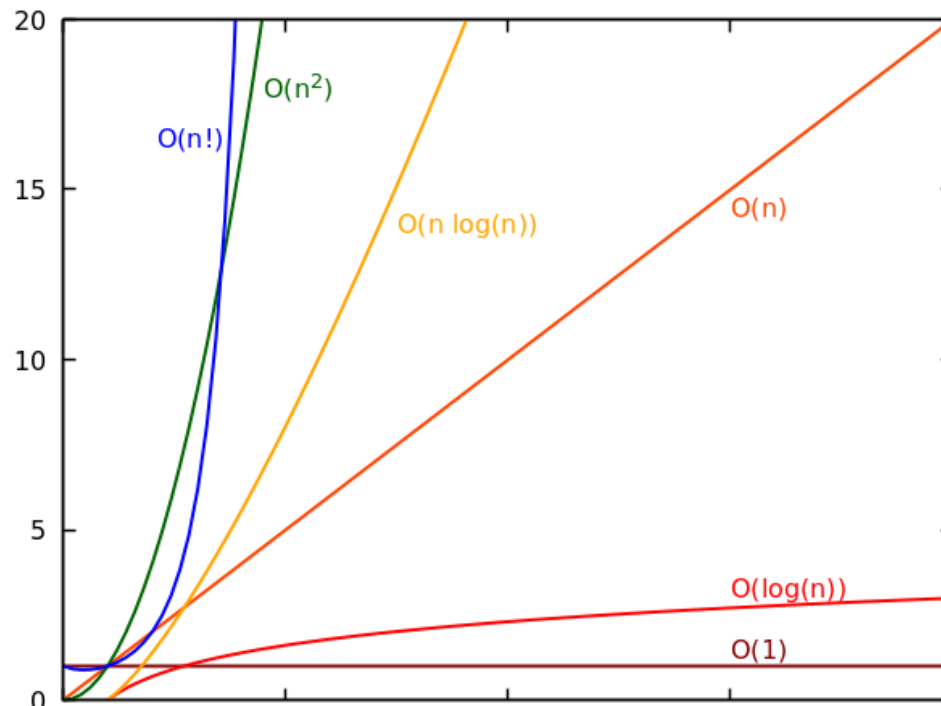
I per poder comparar fàcilment molts algorismes diferents farem una aproximació dels passos computacionals, i només ens fixarem en els termes que aporten més complexitat a l'algorisme:

- **Ometrem constants multiplicatives**: $14n^2$ serà n^2
- Si el cost està expressat com a $\log(n^a) + n^b$ **descartarem el logaritme i només considerarem el polinomi n^b**
- Si el cost està expressat com a $n^a + n^b$ **només considerarem l'índex major**, si $a > b$, n^a
- Si el cost està expressat com a $3^n + 2^n + n^b$ **només considerarem l'exponencial més gran**, 3^n

Vegeu el notebook Complexitat.ipynb per entendre com calcular els passos computacionals.

Complexitat: Gran O – Representació visual

La Gran O ens permetrà agrupar els algoritmes en famílies i veure com evolucionen quan la n creix.



Constant	Logarítmic	Lineal	Superlineal	Quadràtic	Cúbic	Exponencial	Factorial
$O(1)$	$O(\log(n))$	$O(n)$	$O(n \log(n))$	$O(n^2)$	$O(n^3)$	$O(c^n)$ amb $c > 1$	$O(n!)$

Complexitat: Gran O, aproximació quantitativa

Si ho mirem en termes absoluts, podem fer algunes afirmacions:

- ❑ A partir de $n = 20$, els algorismes **factorials** $O(n!)$ ja no tenen sentit
- ❑ Els algorismes **exponencials** $O(C^n)$ són inútils a partir de $n = 40$
- ❑ Els algorismes **quadràtics** $O(n^2)$ comencen a ser costosos a partir de $n = 10.000$ i a ser inútils a partir de $n = 1.000.000$
- ❑ Els algorismes **lineals** $O(n)$ o **super-lineals** $O(n \log n)$ poden arribar fins a $n = 1.000.000.000$
- ❑ Els algorismes **logarítmics** $O(\log n)$ i **els constants** $O(1)$, són útils per a qualsevol n .

n	n^2	$n!$
5	25	120
6	36	720
7	49	5.040
8	64	40.320
9	81	362.880
10	100	3.628.800

El cost de treballar amb cadenes de text

Quan treballem amb cadenes de text, qualsevol operació es complica perquè hem de fer el tractament caràcter a caràcter.

Veiem per exemple el següent codi:

```
def is_unique(paraula):  
    characters_vistos = []  
    for car in paraula:  
        if car in characters_vistos:  
            return False  
        characters_vistos.append(car)  
    return True
```

El cost de treballar amb cadenes de text (II)

```
def is_unique(paraula):  
    characters_vistos = []           # 1 pas  
    for car in paraula:             # per n, on n = nombre caràcters a paraula  
        if car in characters_vistos: # in té un cost m, m = len (characters_vistos)  
            return False           # 1 pas  
        characters_vistos.append(car) # 1 pas  
    return True
```

Caràcters _vistos tindrà 1 caràcter el primer cop, 2 el segon, 3... fins a n-1 caràcters en el pitjor cas. El cost d'iterar sobre aquest in és una sèrie aritmètica:

$$\sum_{i=1}^q i = \frac{q(q+1)}{2} \approx q^2$$

El cost és $1 + n^2 + n + 1 \rightarrow O(n^2)$

Algorismes recursius - definició

Alguns algorismes es criden a sí mateixos en el codi.

És el que s'anomena **algorismes recursius**.

```
def factorial(n: int) -> int:  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

En aquest exemple, si fem la crida:

factorial(5)

S'executarà factorial(4), factorial(3), factorial(2), factorial(1) i a factorial(0)

La recursivitat **ajuda a conceptualitzar** un problema de manera elegant.

Algorismes recursius

En els algorismes recursius hi ha un cost de memòria, la pila d'execució.

```
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

- Per a cada execució l'ordinador guarda el context de la funció.
- L'execució no sempre és eficient, si hi ha opció iterativa **cal valorar** avantatges i inconvenients tenint molt en compte l'eficiència.

Què està passant dins l'ordinador?

```
# recursiu
def factorial(n: int) -> int:
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

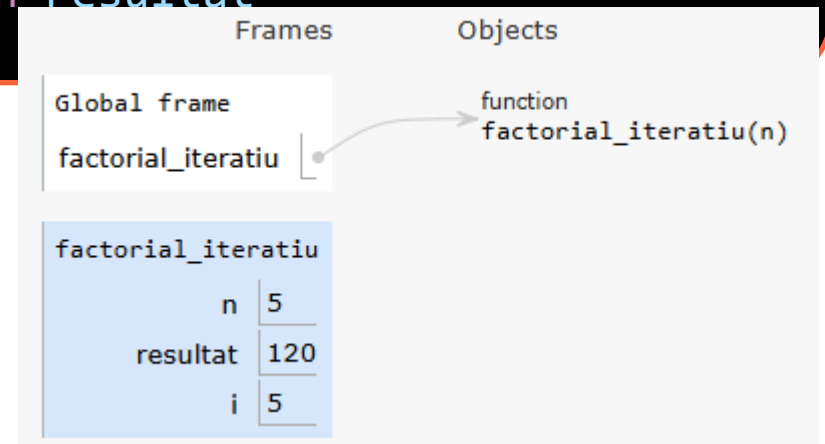
```
# iteratiu
def factorial_iteratiu(n: int) -> int:
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat
```

Què està passant dins l'ordinador? – la pila d'execució

```
# recursiu
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```



```
# iteratiu
def factorial_iteratiu(n: int) -> int:
    resultat = 1
    for i in range(1, n + 1):
        resultat *= i
    return resultat
```



L'estratègia: dividir, vèncer i combinar

Dividir i vèncer (*divide and conquer*) és una estratègia de resolució de problemes que consisteix en:

- ❑ **Dividir** un problema en un o més subproblemes que són instàncies del mateix problema amb una entrada més petita.
- ❑ **Vèncer**: Amb les divisions arribem a un problema que sabem resoldre. Hem vençut el problema!
- ❑ **Combinar** adequadament les solucions dels subproblemes per trobar la solució del problema original. En el cost de combinar també considerem la preparació de les dades dels subproblemes.

L'esquema general és: Tenim un problema de mida n , que reformulem mitjançant la solució d' a problemes de mida n/b i llavors combinem les respostes en un temps $O(n^d)$

El Teorema Mestre (Master Theorem)

Tenim un problema de mida n , que reformulem mitjançant la solució d'a problemes de mida n/b i llavors combinem les respostes en un temps $O(n^d)$.

La complexitat d'aquest tipus de problema es resol pel *Teorema Mestre*:

$$T(n) = aT(n/b) + O(n^d)$$

Si $T(n) = aT(n/b) + O(n^d)$ per algunes constants $a > 0$, $b > 1$, i $d \geq 0$, llavors:

- Cas 1: $T(n) = O(n^d)$ si $a < b^d$, domina el pes de la combinació,
- Cas 2: $T(n) = O(n^d \log n)$ si $a = b^d$, ambdós pesos estan equilibrats,
- Cas 3: $T(n) = O(n^{\log_b a})$ si $a > b^d$, domina el pes dels subproblemes.

Aplicació del Teorema Mestre en un exemple

Imaginem un cas en el que: n és la mida de la llista,

- a , el nombre de subproblemes, és 2,
- b , el divisor de l'entrada és 2 i
- d , l'exponent del cost de preparar i combinar les solucions és 0.

$a > b^d$, $2 > 2^0$, per tant estem en el cas 3, i $T(n) = O(n^{\log_b a})$, és a dir

$$T(n) = O(n^{\log_2 2}), \text{ } T(n) = O(n).$$

Altres exemples del Teorema Mestre

a	b	d	Cas Teorema Mestre	T(n)	O()
3	4	2	$a < b^d \rightarrow$ Cas 1 (combinació)	$O(n^d)$	$O(n^2)$
2	2	1	$a = b^d \rightarrow$ Cas 2 (iguals)	$O(n^d \log n)$	$O(n \log n)$
8	2	2	$a > b^d \rightarrow$ Cas 3 (subproblemes)	$O(n^{\log_b a})$	$O(n^{\log_2 8}) = O(n^3)$

Aquest teorema l'usarem repetidament en els algorismes del tema d'ordenació i cerca.

Ara et toca...



Copia el codi de les solucions del tema 1 demanar números i factorial, i calcula'n els passos computacionals i la complexitat O gran.

Gràcies i recapitulació



MIREIA RIBERA | ACCESSIBILITAT DIGITAL
EXPERIÈNCIA D'USUARI
VISUALITZACIÓ DE DADES



UNIVERSITAT DE
BARCELONA



Recapitulació

- ❑ Per saber l'eficiència d'un algorisme ens cal **saber calcular el seu cost**.
- ❑ Cal tenir especial **cura** en **algorismes recursius**, amb **cadena de text** i amb **col·leccions**. La **col·lecció diccionari** presenta una complexitat **òptima** en l'accés a una dada concreta.
- ❑ A algorísmica fem una aproximació a la complexitat amb **O gran**. El que ens interessa és saber **com creix el cost en funció de la mida de l'entrada**.
- ❑ Si el nostre algorisme usa l'estratègia de dividir i vèncer calcularem la complexitat amb el **teorema Mestre**.



Recursos adicionales

- Introduction to Big O Notation and Time Complexity
[<https://www.youtube.com/watch?v=D6xkbGLQesk>]
- Python Time Complexity [<https://wiki.python.org/moin/TimeComplexity>]